Call Me Maybe

Cours 1 — Les LLMs et la génération de texte

---

1. Qu'est-ce qu'un LLM ?

Un **Large Language Model (LLM)** est un programme entraîné sur une quantité astronomique de texte (milliards de pages web, livres, articles). Son unique objectif d'entraînement est de répondre à une seule question :

> ***« Étant donné ce texte, quel mot vient le plus probablement ensuite ? »***

La « magie » vient du fait que répondre correctement à cette question des millions de fois de suite produit quelque chose qui ressemble à de la compréhension, du raisonnement, de la créativité.

2. Comment il génère du texte ?

Un LLM ne produit **pas** toute une phrase d'un coup. Il génère un mot (ou un fragment de mot) à la fois, en boucle. À chaque étape, le mot choisi s'ajoute au texte existant, qui devient la nouvelle entrée.

1	**Prompt initial** Le texte de départ fourni par l'utilisateur. Ex : "La nuit, tous les"
2	**Tokenisation** Le texte est découpé en sous-mots (tokens). Ex : ["La", " nuit", ",", " tous", " les"]
3	**Conversion en IDs** Chaque token est remplacé par un entier. Ex : [1426, 7012, 11, 2395, 514]
4	**Traitement par le réseau** Le modèle traite ces nombres à travers des milliards de paramètres.
5	**Logits en sortie** Le modèle produit un score (logit) pour chacun des ~150 000 tokens du vocabulaire.

6	**Sélection du token suivant** On choisit le token avec le score le plus élevé (argmax) ou selon une distribution.
7	**Répétition** Le token choisi s'ajoute au texte et on repart à l'étape 1 — jusqu'au token [FIN].

Exemple concret :

Étape	Texte courant (entrée)	Token prédit
Étape 1	`"La nuit, tous les"`	→ **chats (42%)**
Étape 2	`"La nuit, tous les chats"`	→ **sont (61%)**
Étape 3	`"La nuit, tous les chats sont"`	→ **gris (38%)**
Étape 4	`"La nuit, tous les chats sont gris"`	→ **[FIN] (72%)**

À chaque étape, le modèle voit TOUT le texte généré jusqu'ici — pas seulement le dernier mot.

3. Les 3 concepts clés

Concept	Description	Exemple

Logits	Scores bruts produits par le modèle pour chacun des ~150 000 tokens du vocabulaire. Un score élevé = token probable.	`token_chat: 8.2`
		`token_chien: 3.1`
		`token_lune: 0.4`

Softmax	Transformation qui convertit les logits en vraies probabilités (entre 0 et 1, qui somment à 100%). C'est une fonction mathématique.	`8.2 → 42% 3.1 → 18% 0.4 → 2% ...`
Argmax	Méthode de sélection la plus simple : on prend le token avec la probabilité la plus haute. Pas de hasard.	`max(42%, 18%, 2%...) → choisit "chat"`

4. Le lien avec ton projet

Dans le projet *call me maybe*, tu vas **intercepter** la génération entre les logits et la sélection du token. C'est le principe du **constrained decoding** :

Mode	Comportement	Résultat
Génération libre	Le modèle choisit n'importe quel token parmi les 150 000 possibles.	→ texte quelconque, JSON souvent invalide
Constrained decoding	Tu mets à -∞ les scores de tous les tokens invalides AVANT la sélection.	→ JSON 100% valide, structuré

Exemple concret : si le modèle a déjà généré `{"name": "fn_add`, le seul caractère valide suivant est _ ou une lettre — pas une accolade, pas un chiffre. Tu mets tous les autres tokens à -∞.

Exercice — Vérifie ta compréhension

Q1. À chaque étape de génération, le modèle voit-il toute la phrase générée jusqu'ici, ou seulement le dernier mot ?

Q2. Si le modèle donne un score de 8.2 à "chats" et 3.1 à "chiens", lequel sera choisi avec la méthode argmax ?

Q3. Dans ton projet, quand le modèle a déjà généré {"name": "fn_add — quels sont les prochains caractères valides ? Lesquels doit-on bloquer ?

Réponses

Q1	Le modèle voit TOUT le texte généré jusqu'ici à chaque étape. C'est pour ça que la phrase reste cohérente : il a le
Q2	argmax choisit le score le plus élevé → "chats" (8.2 > 3.1).
Q3	Caractères valides : lettres (a-z, A-Z), chiffres, underscore _ (pour compléter le nom de fonction). Bloquer : accola

---

A retenir

- Un LLM prédit un token à la fois, en boucle.
- À chaque étape, le modèle voit tout le texte précédent.
- Les logits sont des scores bruts → softmax → probabilités → sélection.
- Le constrained decoding met certains logits à -∞ pour forcer une structure.

Prochaine étape

Cours 2 — La tokenisation : comprendre comment le texte est découpé en tokens, ce qu'est un vocabulaire, et pourquoi les espaces et la casse ont de l'importance pour le constrained decoding.